# Extended Lab Task: The PixelForge Sprint Crisis

*A Multi-Phase Group Git Workflow Simulation (Follow-On to the Git Workflow Management Lab)*

Group Size: 4 students | Duration: 2 lab sessions (recommended)

## 1. Objective

This task extends the basic Git Workflow Management simulation into a longer, higher-stakes scenario. Instead of one clean feature branch per student, your team of four will run a full sprint that includes two features developed in parallel that deliberately collide, an urgent production hotfix that cannot wait for the normal review cycle, and a formal release with semantic versioning. The goal is not just to execute commands correctly, but to make judgment calls under the same kind of pressure real teams face: whose change wins when two people edit the same lines, and how do you patch production without losing the fix on the next release.

## 2. The Story So Far

Your team is PixelForge Studios. Two sprints ago you shipped the first version of TaskZen, a productivity app, and tagged it v1.0.0. main, dev, and stage exist exactly as set up in the original lab. The login page feature (T-14) is already live in production. This week, the product owner has posted a new batch of Trello cards — and, unfortunately, a user has just reported a bug in the feature you already shipped.

## 3. Team Roles

Assign one role per member for the duration of this task. Roles rotate on the next group assignment, not mid-sprint — part of the exercise is experiencing the responsibility that comes with a single role under pressure.

| Role | Assigned To | Core Responsibility This Sprint |
|---|---|---|
| **Integration Lead** | Student A | Owns dev and release/* branches. Cuts the release branch, promotes stage → main, re-syncs dev, deletes merged branches. |
| **Feature Developer 1** | Student B | Implements T-15 (Dashboard Analytics Widget). First to open a PR against dev. |
| **Feature Developer 2** | Student C | Implements T-16 (Notification Center) in parallel with Student B, on purpose overlapping the same file section — will need to resolve the resulting conflict. |
| **QA / Release Reviewer** | Student D | Reviews all PRs, signs off on stage before promotion to main, responds to the Phase 3 hotfix as first responder. |

## 4. Phase 1 — Parallel Feature Development (Days 1–2)

Student B and Student C each claim a new Trello card and branch from dev at the same starting commit. Both features are described as appending a new subsection under the same '## Core Features' heading in PROJECT_FEATURES.md — this overlap is intentional.

### Student B — T-15: Dashboard Analytics Widget

Add the following to the end of PROJECT_FEATURES.md:

```
### T-15: Dashboard Analytics Widget
Adds a chart widget to the dashboard summarising tasks completed per week.
**Status: Implemented**
```

**Task:** Starting from an up-to-date dev branch, create a correctly-named feature branch, add the content above to PROJECT_FEATURES.md, then stage, commit with a properly formatted message, and push the branch so a PR can be opened.

***Hint:*** *You'll need, in some order: git checkout, git pull, git checkout -b, git add, git commit -m, git push -u origin <branch-name>.*

Student B opens a PR into dev immediately. Because no one else has merged yet, this PR merges cleanly.

### Student C — T-16: Notification Center (started in parallel, before B's merge)

Add the following to the end of PROJECT_FEATURES.md:

```
### T-16: Notification Center
Adds an in-app notification centre with read/unread state and grouping by task.
**Status: Implemented**
```

**Task:** At the same time as Student B — without waiting for their PR — branch from dev, add the content above, and push your own feature branch so a PR can be opened.

***Hint:*** *Same command set as Student B. The important part is the timing: start before B's branch is merged, not after.*

Student C opens a PR into dev after Student B's has already merged. Because both branches appended to the exact same location in the file, GitHub/GitLab will report the PR as unable to merge automatically — this is your conflict.

## 5. Phase 2 — Resolving the Merge Conflict

This phase is done individually by Student C, with Student D available for consultation, not resolution — the person who caused the conflict is the one who must fix it.

### Step 5.1: Bring dev's new state into the feature branch

**Task:** While on feature/T-16, pull in dev's latest state (which now includes B's merged T-15) so the conflict shows up locally instead of only in the PR's web UI.

***Hint:*** *You'll need: git fetch, then a command that brings origin/dev's commits into your current branch. Git will report something like 'CONFLICT (content): Merge conflict in PROJECT_FEATURES.md' — that's expected.*

### Step 5.2: Open the file and resolve manually

The file will contain conflict markers around the disputed section:

```
<<<<<<< HEAD
### T-16: Notification Center
```

```
Adds an in-app notification centre with read/unread state and grouping by task.
**Status: Implemented**
=======
### T-15: Dashboard Analytics Widget
Adds a chart widget to the dashboard summarising tasks completed per week.
**Status: Implemented**
>>>>>>> origin/dev
```

Edit the file by hand so that BOTH sections survive, one after another, with the conflict markers removed entirely. Do not simply keep one side — losing a teammate's merged work here is treated as a serious process failure, not a shortcut.

#### Step 5.3: Mark resolved, commit, and push

**Task:** Tell Git the conflict is resolved, complete the merge commit, and push the branch.

***Hint:*** *You'll need: git add on the resolved file, then a commit command (Git will pre-fill a merge commit message for you — keep or edit it), then git push.*

Student D reviews the resolved PR specifically for correctness of the merge (both sections present, no stray conflict markers left in the file) before approving and merging into dev — use the platform's regular merge option here, NOT Squash and Merge, so the conflict-resolution commit is preserved in history.

**Task:** Update your local dev branch and delete both feature branches locally now that they're merged.

***Hint:*** *git checkout, git pull, and a branch-deletion command run twice (once per branch).*

## 6. Phase 3 — Production Hotfix Crisis (Day 3)

Mid-sprint, a new Trello card appears in a special 'URGENT — Production' swimlane: "T-21: Login page shows the wrong error message on invalid password (regression from T-14)." This bug is live in main right now, so it cannot wait for dev to be released — you can't ship two half-finished features (T-15/T-16 are still in dev, not yet released) just to fix a one-line bug.

#### Step 6.1: Branch the hotfix from main, not dev

Add the following to the end of PROJECT_FEATURES.md:

```
### T-21: Fix Login Error Message (hotfix)
Corrects the login form to show 'Incorrect email or password' instead of a raw server
error code.
**Status: Fixed**
```

**Task:** Decide which branch this hotfix must start from (think about what's actually running in production right now), create a correctly-named hotfix branch from it, add the content above, commit, and push.

***Hint:*** *Same command set as a feature branch — the thing to get right here is the starting branch, not the commands themselves.*

#### Step 6.2: Emergency review and merge into main

Student D reviews as fast as possible — this PR skips the normal backlog and is treated as top priority. Base: main. Compare: hotfix/T-21.

**Task:** After PR approval, update your local main with the merge, then create an annotated tag marking this as v1.0.1 and push the tag to origin.

***Hint:*** *git checkout, git pull, then an annotated tag command (look for the flag that lets you attach a message to a tag) — note that tags don't travel with a normal 'git push', they need pushing explicitly by name.*

### Step 6.3: Do not lose the fix — merge it into dev as well

A hotfix that only lives in main will be silently overwritten the next time dev is released, reintroducing the bug. Merge the same fix into dev immediately.

**Task:** Switch to dev, make sure it's current, bring main's commits (including the hotfix) into it, push, and delete the now-merged hotfix branch locally.

***Hint:*** *git checkout, git pull, git merge <branch>, git push, and a branch-deletion command.*

**Checkpoint:** at this point dev contains T-15, T-16, and the T-21 hotfix. main contains only the T-21 hotfix on top of the original v1.0.0. This divergence is expected and is the entire point of the exercise.

## 7. Phase 4 — Release Branch and Semantic Versioning (Day 4)

Student A (Integration Lead) now packages the completed features for release.

### Step 7.1: Cut the release branch from dev

**Task:** From an up-to-date dev, create a release branch named release/v1.1.0 and push it upstream.

***Hint:*** *git checkout, git pull, git checkout -b, git push -u origin <branch-name>.*

### Step 7.2: Promote to stage for QA sign-off

**Task:** Bring the release branch's commits into stage and push.

***Hint:*** *git checkout stage, git pull, git merge <branch>, git push.*

Student D performs QA sign-off by commenting on the corresponding Trello release card, listing exactly which task IDs (T-15, T-16, T-21) are present and confirming PROJECT_FEATURES.md contains all three sections correctly. Only after this comment is posted may the release proceed to main.

### Step 7.3: Promote stage to main and tag the release

**Task:** Bring the release branch's commits into main, create an annotated v1.1.0 tag, and push both the branch update and the tag to origin.

***Hint:*** *Same merge-and-tag pattern as Step 6.2, but the base branch is main and the merge source is release/v1.1.0.*

### Step 7.4: Re-sync dev and clean up

**Task:** Bring main's latest state (with the release now included) back into dev, push, then delete the release branch both on origin and locally.

***Hint:*** *git checkout, git pull, git merge, git push, then a remote branch-deletion command (different syntax from a local one) plus a local one.*

Note the version jump: v1.0.0 → v1.0.1 (patch, hotfix only) → v1.1.0 (minor, new features). If T-21 had been a breaking change instead of a bug fix, the appropriate next version would follow semantic versioning's MAJOR.MINOR.PATCH convention accordingly — this distinction should be explained in your retrospective.

## 8. Phase 5 — CHANGELOG and Retrospective (Day 4–5)

### Step 8.1: Update CHANGELOG.md on main

Following Keep a Changelog conventions, add entries for both releases (do this as its own small commit on main, then re-sync into dev as in Step 7.4):

```
## [1.1.0] - <date>
### Added
- Dashboard analytics widget (T-15)
- Notification center (T-16)

## [1.0.1] - <date>
### Fixed
- Login page showed a raw server error instead of a friendly message (T-21)
```

### Step 8.2: Individual contribution log

**Task:** Each member produces a one-line-per-commit log of only their own commits on the final dev branch, and includes the output in the group submission.

***Hint:*** *git log has a flag to filter by author and a flag to condense each commit to one line — combine both, targeting the dev branch.*

### Step 8.3: Written retrospective (half a page per group, not per person)

Answer, in your own words:

- What would have happened if Student C had resolved the Phase 2 conflict by force-pushing over Student B's merged work instead of merging both sections?
- Why was the Phase 3 hotfix branched from main instead of from dev?
- Why did the fix have to be merged into dev separately in Step 6.3, and what bug would have resurfaced later if that step had been skipped?
- Explain, using this sprint's actual tags, the difference between the PATCH and MINOR parts of semantic versioning.

## 9. Command Toolbox (Reference, Not a Sequence)

Every command you'll need across all five phases is listed below, in no particular order. Matching the right command to the right task — and the right order within a task — is the point of the exercise.

- git checkout <branch>
- git checkout -b <branch>
- git pull origin <branch>
- git push -u origin <branch>
- git push origin <branch>
- git fetch origin
- git merge <branch>
- git merge origin/<branch>
- git add <file>
- git commit -m "<message>"

- git commit (no -m, to accept/edit an auto-filled merge message)
- git tag -a <tag> -m "<message>"
- git push origin <tag>
- git branch -d <branch> (delete local branch)
- git push origin --delete <branch> (delete remote branch)
- git log --author="<name>" --oneline <branch>
- git status (use this liberally to check what state you're in before and after every step)

## 10. Deliverables

- Repository URL, shared by the Integration Lead.
- Screenshot of the tag list showing v1.0.0, v1.0.1, and v1.1.0.
- Each member's individual contribution log (Step 8.2).
- Final PROJECT_FEATURES.md, CHANGELOG.md, and the group retrospective (Step 8.3), submitted as a single PDF or shared doc link.

## 11. Grading Rubric

| Criterion | Excellent (Full Marks) | Good (Partial) | Needs Improvement |
|---|---|---|---|
| **Branch naming & structure** | All feature/hotfix/release branches follow convention exactly; branched from correct base | Minor naming inconsistency or one branch off the wrong base | Branches missing, wrongly named, or created off wrong parent |
| **Commit message convention** | Every commit follows type(scope): summary + meaningful body where required | Mostly consistent, 1–2 messages vague | Messages missing type prefixes or uninformative (e.g. 'update') |
| **Merge conflict resolution (Phase 2)** | Conflict resolved manually with both features intact, no data loss, clean commit history | Conflict resolved but one feature partially overwritten or history messy | Conflict resolved by discarding a teammate's work, or resolved by force-push |
| **Hotfix workflow (Phase 3)** | Hotfix branched from main, merged into BOTH main and dev, tagged correctly | Hotfix merged into main only, or tag missing/incorrect | Hotfix built from dev instead of main, or bug left unresolved in one branch |
| **Release & tagging (Phase 4)** | Release branch correctly promoted stage → main, semantic version tags applied, dev re-synced | Release completed but versioning inconsistent or dev not synced back | No formal release branch/tag used, or main updated without going through stage |
| **CHANGELOG accuracy** | CHANGELOG.md follows Added/Fixed/Changed format, matches actual commits and tags | CHANGELOG present but incomplete or slightly mismatched with history | CHANGELOG missing or not updated for the release |
| **Individual contribution evidence** | git log --author output clearly shows each member's distinct, proportionate contribution | Contributions visible but uneven or hard to attribute | One or more members show no independent commits |

| | | | |
|---|---|---|---|
| **Code review quality** | PR comments show genuine review (naming, message format, content) before approval | PRs approved with minimal comments | PRs approved without any visible review activity |

**Note:** *individual grades within the group may be adjusted based on the contribution log — a member with no independent commits across all five phases will be graded on documented participation only.*